DATA MODELLING

# Multi-Tenant Appointment Booking Platform

*Initial logical data model, entity rationale, relationships, integrity rules and screen-to-data traceability*

**Presentation purpose** Explain what the system must remember, why each entity exists, how records are connected, and how the model supports booking without double-booking.

Prepared from the frozen Project System Requirements & Planning Document v2.1

Data-modelling scope only • Wireframes maintained separately • 28 August 2026

# 1. Modelling Scope and Assumptions

This model supports a web-based SaaS platform in which isolated service businesses publish services, assign eligible providers, configure provider availability and accept customer appointments. It covers both multi-staff organisations and a business-of-one.

| Confirmed rule | Data-model consequence |
|---|---|
| Multi-tenancy | Every operational record belongs to one Business; queries and authorization are tenant-scoped. |
| Business-of-one | A single User can hold both ADMIN and PROVIDER roles; the provider identity still counts toward capacity. |
| Fixed service duration | Duration is stored once on Service and used to derive appointment end time and available slots. |
| No payments in MVP | No Payment, Invoice or Refund entities are included. |
| Individual calendars | Availability belongs to Provider Profile; rooms and equipment are out of scope. |
| Four appointment statuses | CONFIRMED, CANCELLED, COMPLETED and NO_SHOW are sufficient for the MVP lifecycle. |

**Modelling principle** Authentication identity, tenant membership and bookable provider identity are separate concepts. This prevents a single-role column from breaking the owner-provider scenario.

# 2. Main Entities and Why They Exist

| Entity | Role in model | Why our system needs it |
|---|---|---|
| Business | Tenant boundary | Keeps each service business operationally isolated. |
| User | Login identity | Stores credentials and reusable personal identity without hard-coding one role. |
| Business Membership | User-to-business link | Represents participation in a tenant and supports membership activation. |
| Membership Role | Authorization assignment | Allows one membership to carry ADMIN, PROVIDER or both roles. |
| Provider Profile | Bookable provider identity | Stores the staff/owner identity used by services, calendars and appointments. |
| Customer Profile | Tenant-specific customer | Connects a login to customer data within a particular business. |
| Service | Bookable offering | Stores the service description, fixed duration and active state. |
| Provider Service | Qualification mapping | Resolves the many-to-many relationship between providers and services. |
| Working Hour | Recurring availability | Stores the provider's standard weekly working windows. |
| Blocked Time | Exceptional unavailability | Stores leave, breaks and time off that remove otherwise valid |

| Entity | Role in model | Why our system needs it |
|---|---|---|
| | | slots. |
| Appointment | Booking transaction | Records the customer, service, provider, time range and current lifecycle status. |
| Appointment Status History | Audit trail | Explains who changed an appointment status and when. |

## 3. Entity Attributes, Keys and Constraints

PK identifies one record. FK represents a relationship to another entity. UK is a uniqueness constraint. UUID keys are recommended to reduce enumeration risk across tenants.

### Business

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Tenant identifier |
| name | VARCHAR(150) | Required | Display name |
| timezone | VARCHAR(64) | Required | Interprets local schedules |
| is_active | BOOLEAN | Required | Soft activation state |
| created_at | TIMESTAMPTZ | Required | Audit timestamp |

### User

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Login identity |
| email | CITEXT | UK, required | Case-insensitive unique login |
| password_hash | TEXT | Required | Never store plain-text passwords |
| full_name | VARCHAR(150) | Required | User display name |
| is_active | BOOLEAN | Required | Authentication access state |
| created_at | TIMESTAMPTZ | Required | Account creation |

### Business Membership

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Membership identifier |
| business_id | UUID | FK → Business | Tenant |
| user_id | UUID | FK → User | Participating user |

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| is_active | BOOLEAN | Required | Membership access state |
| joined_at | TIMESTAMPTZ | Required | Audit timestamp |

## Membership Role

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| membership_id | UUID | PK/FK → Membership | Role owner |
| role | ENUM | PK | ADMIN or PROVIDER |

## Provider Profile

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Bookable provider |
| business_id | UUID | FK → Business | Tenant scope |
| membership_id | UUID | FK → Membership, UK | One provider identity per membership |
| display_name | VARCHAR(150) | Required | Customer-facing name |
| is_active | BOOLEAN | Required | Counts toward active-provider limit |

## Customer Profile

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Customer identifier |
| business_id | UUID | FK → Business | Tenant scope |
| user_id | UUID | FK → User | Account owner |
| phone | VARCHAR(30) | Optional | Contact detail |
| is_active | BOOLEAN | Required | Soft activation state |

## Service

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Service identifier |
| business_id | UUID | FK → Business | Tenant scope |
| name | VARCHAR(150) | Required | Catalog label |
| description | TEXT | Optional | Customer-facing details |

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| duration_minutes | INTEGER | Required, > 0 | Fixed MVP duration |
| is_active | BOOLEAN | Required | Controls customer visibility |

## Provider Service

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| provider_id | UUID | PK/FK → Provider | Eligible provider |
| service_id | UUID | PK/FK → Service | Eligible service |
| is_active | BOOLEAN | Required | Reversible assignment |

## Working Hour

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Availability row |
| provider_id | UUID | FK → Provider | Calendar owner |
| weekday | SMALLINT | 0–6 | Day of week |
| start_time | TIME | Required | Opening boundary |
| end_time | TIME | Required; end > start | Closing boundary |

## Blocked Time

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Blocked period |
| provider_id | UUID | FK → Provider | Calendar owner |
| start_at | TIMESTAMPTZ | Required | Block begins |
| end_at | TIMESTAMPTZ | Required; end > start | Block ends |
| reason | VARCHAR(200) | Optional | Leave/break context |

## Appointment

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Appointment identifier |
| business_id | UUID | FK → Business | Tenant scope |
| customer_id | UUID | FK → Customer | Booking owner |

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| provider_id | UUID | FK → Provider | Assigned calendar |
| service_id | UUID | FK → Service | Booked offering |
| start_at | TIMESTAMPTZ | Required | Inclusive start |
| end_at | TIMESTAMPTZ | Required | Exclusive end |
| status | ENUM | Required | CONFIRMED/CANCELLED/COMPLETED/NO_SHOW |
| created_at / updated_at | TIMESTAMPTZ | Required | Audit timestamps |

## Appointment Status History

| Field | Type | Key / rule | Purpose |
|---|---|---|---|
| id | UUID | PK | Audit record |
| appointment_id | UUID | FK → Appointment | Changed appointment |
| old_status / new_status | ENUM | Required | Transition |
| changed_by_user_id | UUID | FK → User | Actor |
| changed_at | TIMESTAMPTZ | Required | Transition time |

## 4. Entity Relationship Diagram

The diagram presents the logical relationships. For readability, secondary tenant links are described in the rules section even when only the primary relationship line is emphasized.

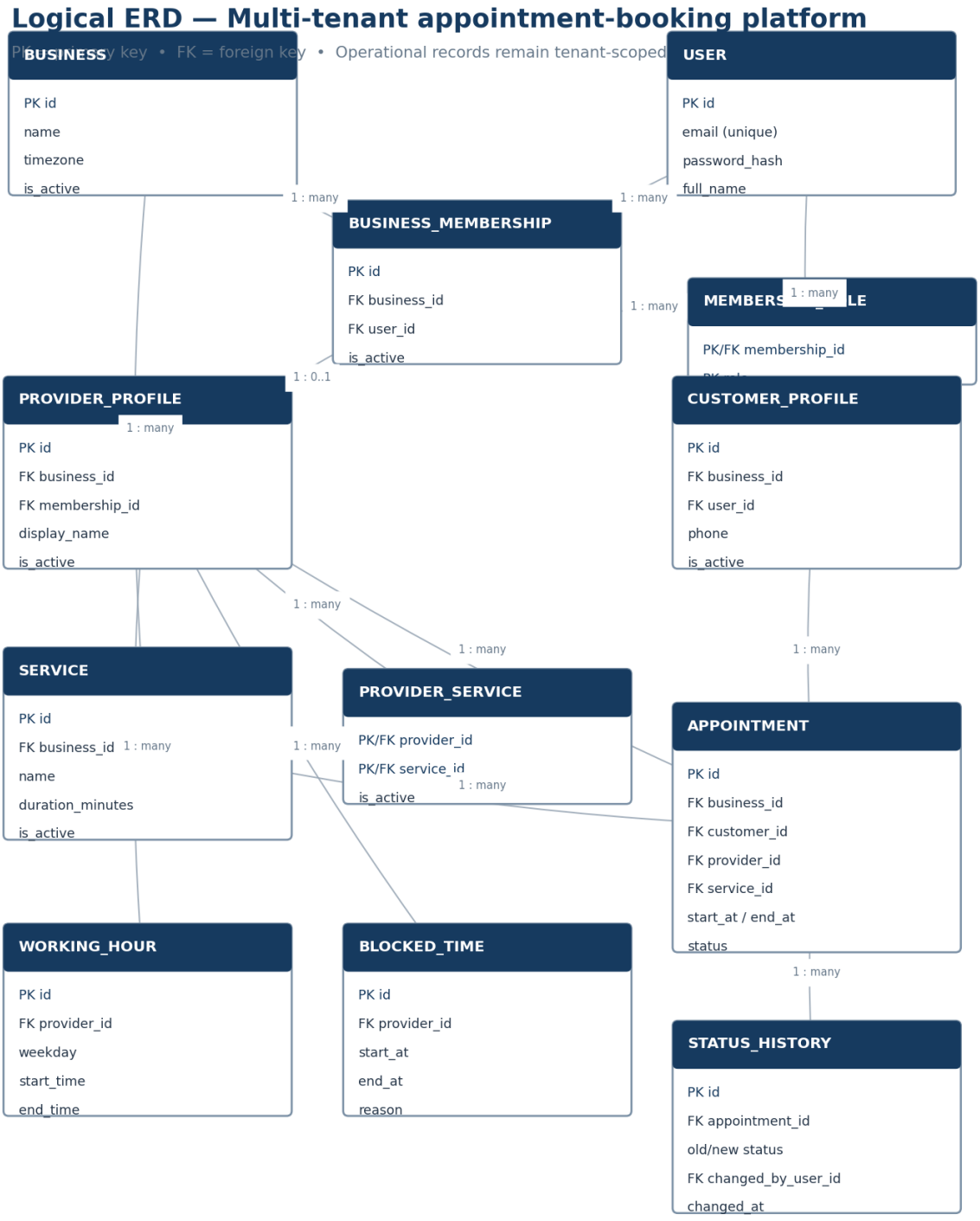



*Figure 1. Proposed logical ERD for the v2.1 MVP.*

## 5. Relationship Explanation and Cardinality

| Relationship | Cardinality | Business justification |
|---|---|---|
| Business → Business Membership | 1 : many | A tenant has multiple participating users; each membership belongs to one tenant. |
| User → Business Membership | 1 : many | A login may participate in more than one business without sharing tenant records. |
| Membership → Membership Role | 1 : many | One membership may be ADMIN, PROVIDER or both. |
| Membership → Provider Profile | 1 : 0..1 | Only memberships acting as a provider need a bookable profile. |
| Business → Service | 1 : many | Services are offered and controlled by exactly one business. |
| Provider ↔ Service | many : many | Providers may perform several services and each service may have several providers; Provider Service resolves the relationship. |
| Provider → Working Hour | 1 : many | A provider may define one or more recurring work windows. |
| Provider → Blocked Time | 1 : many | A provider may create multiple exceptional periods of unavailability. |
| Customer → Appointment | 1 : many | A customer may book many appointments; each appointment has one booking owner. |
| Provider → Appointment | 1 : many | A provider receives many appointments over time; each appointment occupies one provider calendar. |
| Service → Appointment | 1 : many | Each appointment is for one service; a service appears in many appointments. |
| Appointment → Status History | 1 : many | Each lifecycle change can be recorded as a separate immutable audit entry. |

## 6. Data Integrity and Booking Rules

- Tenant integrity: the appointment, customer, provider and service must all belong to the same Business.
- Qualification integrity: a booking is valid only when an active Provider Service mapping exists.
- Availability integrity: the full appointment range must fall inside a provider working window and outside blocked time.
- Conflict integrity: two CONFIRMED appointments for one provider must never overlap.
- Lifecycle integrity: only CONFIRMED appointments can transition; CANCELLED, COMPLETED and NO_SHOW are terminal.
- Time integrity: customers may cancel or reschedule only while current time is strictly before start_at.
- Provider status integrity: a provider may set COMPLETED or NO_SHOW only for their own appointment at or after start_at.
- Concurrency integrity: booking and rescheduling must revalidate and secure the slot in one transaction.

**Overlap test** A conflict exists when new_start < existing_end AND new_end > existing_start. Adjacent appointments are therefore allowed, while intersecting ranges are rejected.

### Recommended PostgreSQL enforcement

Use application validation for user-friendly feedback and database enforcement as the final safety boundary. A partial exclusion constraint on the provider and timestamp range can reject overlapping CONFIRMED appointments even under concurrent requests.

## 7. Appointment Lifecycle

| Current state | Permitted action | Condition | Result |
|---|---|---|---|
| — | Create booking | Valid and atomically secured slot | CONFIRMED |
| CONFIRMED | Reschedule | Customer owns record and now < start_at | CONFIRMED with new start/end |
| CONFIRMED | Cancel | Customer owns record and now < start_at | CANCELLED; slot released |
| CONFIRMED | Complete | Authorized provider/admin and now ≥ start_at | COMPLETED terminal |
| CONFIRMED | Mark no-show | Authorized provider/admin and now ≥ start_at | NO_SHOW terminal |
| CANCELLED / COMPLETED / NO_SHOW | Any further status transition | Never permitted | Rejected |

## 8. Screen → Action → Backend → Data

Wireframes are maintained separately. This traceability view demonstrates that the planned screens and the data model describe the same system, as required by the mentor guide.

| Screen / area | User action | Backend process | Data affected |
|---|---|---|---|
| Service catalogue | View active offerings | Filter Service by business_id and is_active | Read Service |
| Provider selection | Choose who performs service | Join Provider Service to active Provider Profile | Read Provider Service + Provider Profile |
| Time-slot picker | Choose date and time | Subtract Blocked Time and confirmed Appointment ranges from Working Hour windows | Read Working Hour, Blocked Time, Appointment |
| Booking confirmation | Reserve selected slot | Revalidate in transaction; calculate end_at from Service duration | Create Appointment as CONFIRMED |
| My appointments | View personal history | Filter Appointment by authenticated Customer Profile | Read Appointment + Service + Provider |

| Screen / area | User action | Backend process | Data affected |
|---|---|---|---|
| Cancel appointment | Cancel eligible booking | Verify ownership, status and cutoff | Update Appointment; append Status History |
| Reschedule appointment | Move eligible booking | Atomically validate new range and update timestamps | Update Appointment; preserve audit history |
| Provider availability | Manage work pattern/time off | Validate ranges and save provider-owned configuration | Create/update Working Hour or Blocked Time |
| Provider schedule | View assigned work | Filter by provider_id and tenant | Read Appointment + Customer + Service |
| Admin service management | Maintain catalogue | Validate tenant ownership and duration | Create/update/deactivate Service |
| Admin provider management | Maintain staff and assignments | Check package capacity; assign membership roles and services | Create/update Membership, Role, Provider Profile, Provider Service |
| Admin master calendar | Manage tenant appointments | Filter all appointments by business_id | Read/update Appointment |

## 8.1 Wireframe-to-Data Evidence

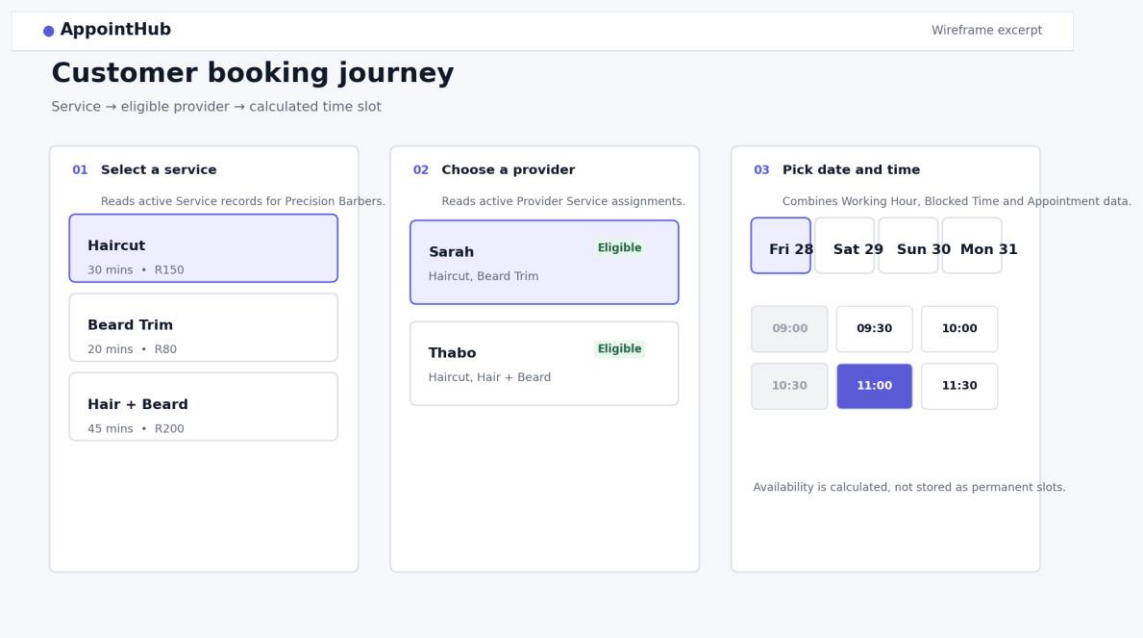



*Figure 2. Customer booking: Service, Provider Service, Provider Profile, Working Hour, Blocked Time and Appointment records collaborate to produce a valid booking choice.*

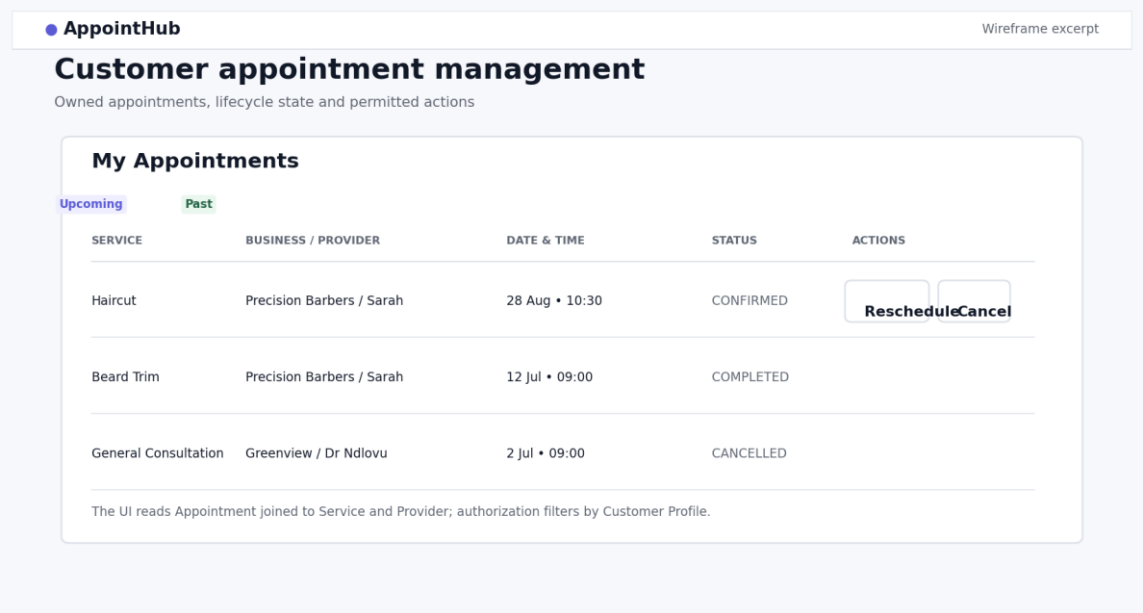



*Figure 3. Customer management: Appointment supplies current state and timing, while Service and Provider Profile supply the descriptive context shown to the customer.*

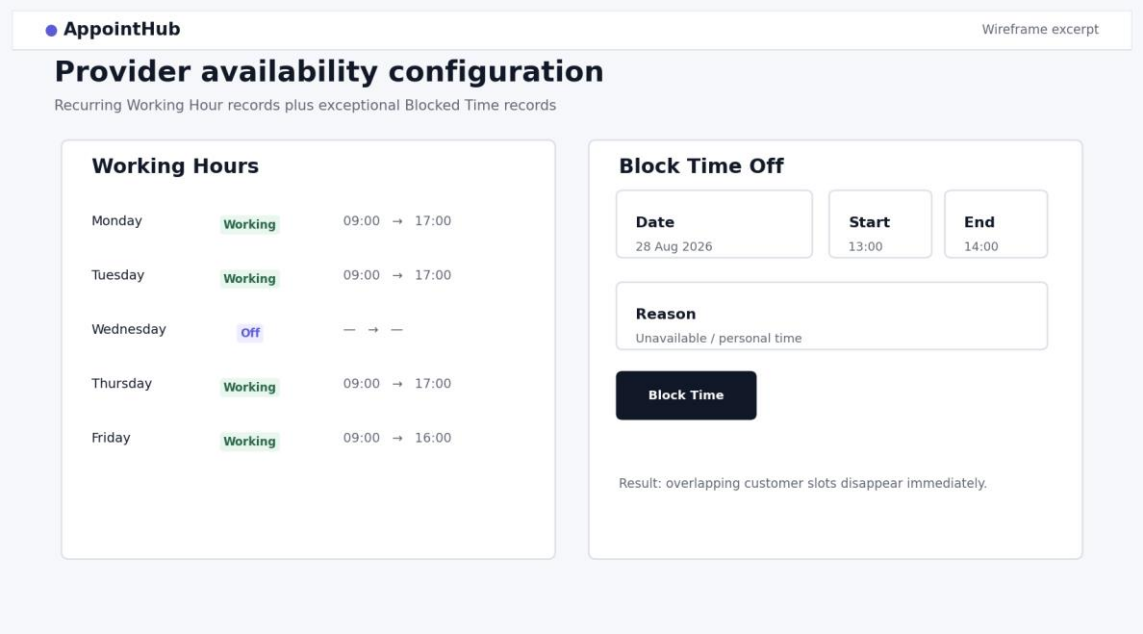



*Figure 4. Provider availability: recurring Working Hour rows establish normal capacity; Blocked Time rows remove exceptional periods from customer availability.*

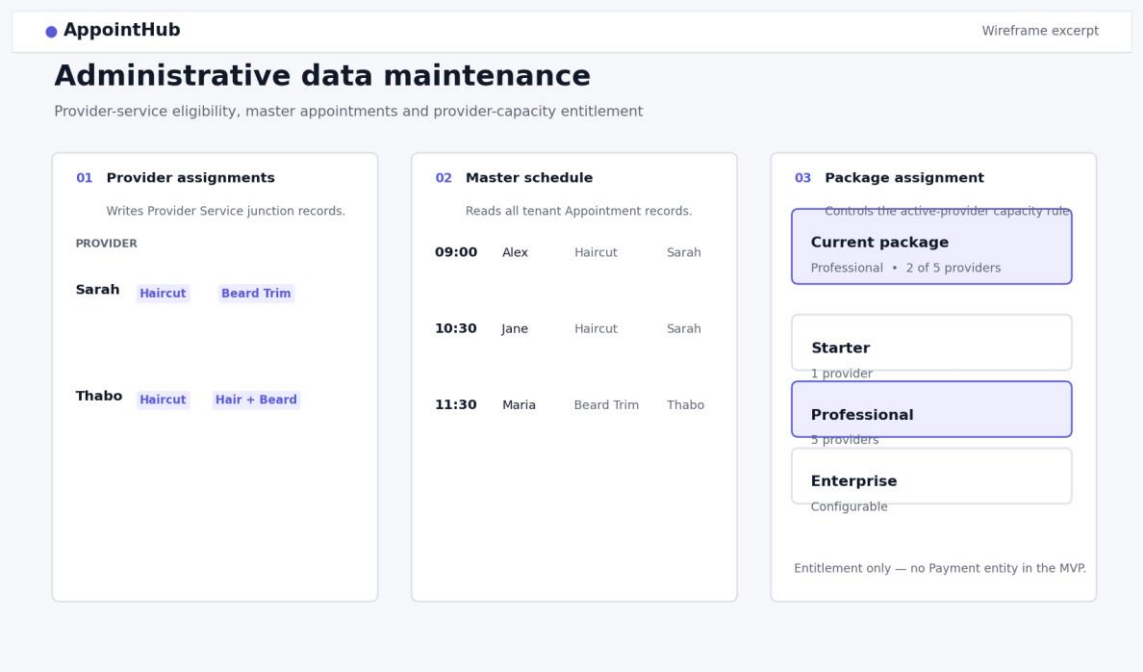



*Figure 5. Administration: Provider Service controls eligibility, Appointment drives the master schedule, and the assigned package controls active-provider capacity.*

## 9. Example Data Flow: Successful Booking

1. Customer selects an active Service. The application reads its duration and tenant.
2. The application reads active Provider Service mappings and presents eligible providers.
3. For the chosen provider and date, it reads Working Hour, Blocked Time and CONFIRMED Appointment records.
4. The customer selects a calculated slot and submits the booking.
5. The server repeats all tenant, qualification, time and overlap checks inside a transaction.
6. The server inserts the Appointment with status CONFIRMED and commits the transaction.
7. The confirmation screen retrieves the stored Appointment together with Service and Provider details.

**Failure path** If another transaction secures the range first, the database rejects the insert. The UI refreshes availability and asks the customer to choose another slot.

## 10. Design Decisions We Can Defend

| Mentor challenge | Defensible response |
|---|---|
| Why not put role on User? | A user may be both administrator and provider, and roles are meaningful within a specific tenant. |
| Why separate Provider Profile from User? | Only the provider identity is bookable and owns availability; authentication details should not be duplicated in scheduling records. |
| Why use Provider Service? | The real business rule is many-to-many qualification, so a junction table is required. |
| Why not store generated slots? | Slots are derived from current working hours, blocked time, service duration and bookings; storing them risks stale availability. |
| Why retain business_id on Appointment? | It makes tenant authorization explicit and supports efficient tenant-wide calendar queries. |
| Why soft-deactivate records? | Historical appointments must remain explainable after a service or provider stops accepting bookings. |
| Why preserve status history? | It provides accountability for customer, provider and administrator actions without changing the current-state model. |

# 11. Open Questions and Future Decisions

| Question | Current modelling position |
|---|---|
| Customer account scope | Confirm whether one global login can book across businesses or whether customer registration is tenant-specific. The proposed model supports global identity with tenant-specific customer profiles. |
| Working-hour complexity | Confirm whether providers need split shifts, overnight shifts or date-specific working-hour overrides. Multiple daily rows support split shifts; overnight ranges may require special handling. |
| Changing service duration | Decide whether Appointment should snapshot duration/service name so historical records remain unchanged after service edits. |
| Provider reassignment | Define whether customers may change provider during rescheduling and which roles can reassign an existing appointment. |
| Admin override limits | Clarify whether administrators may override working hours/blocked time or must still obey conflict rules. Double-booking prevention should remain absolute. |
| Package data | Confirm whether package/provider limits are stored locally on Business/Subscription or supplied by a separate billing service. |
| Deletion policy | Agree retention and anonymisation rules for customer accounts and appointment history. |

## 12. Mentor Review Checklist

- ☐ Main entities come directly from the appointment-booking requirements.
- ☐ Every major entity has a clear business reason for existing.
- ☐ Important attributes, primary keys and foreign keys are identified.
- ☐ One-to-one, one-to-many and many-to-many relationships are explained.
- ☐ The ERD represents multi-tenancy and the business-of-one model.
- ☐ The model supports provider eligibility, availability and conflict prevention.
- ☐ Appointment status and time-based transition rules are explicit.
- ☐ Each planned screen/action has an identifiable data source or destination.
- ☐ Errors and concurrent booking conflicts have a defined data-level response.
- ☐ Uncertain or future decisions are listed rather than hidden.

**Presentation close** This is our current logical model for the MVP. It is detailed enough to guide API and database design, while the open questions identify where mentor or client confirmation may still change implementation.